

DEPARTMENT OF INFORMATION TECHNOLOGY

24ITT32 - PROGRAMMING IN JAVA

INDIVIDUAL PROJECT ON

ACADEMIC MANAGEMENT SYSTEM

P R O J E C T R E P O R T

Submitted by

VAISHNAVI SRIDHAR MAHESWARI

25ITR166

JAVA ASSIGNMENT

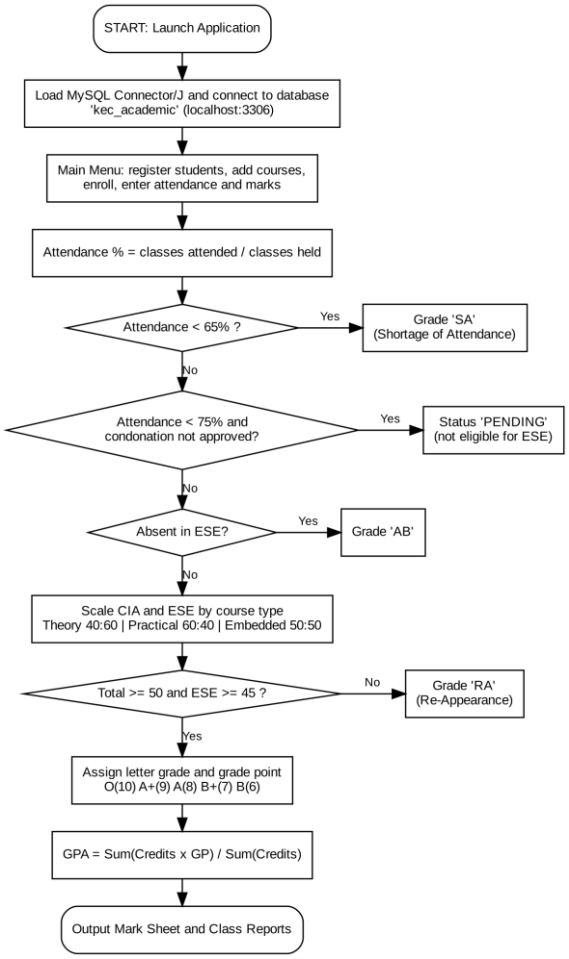
AIM:

Development of Kongu Engineering College (R2024) Student Academic & Continuous Evaluation System using Java JDBC and MySQL.

OBJECTIVE:

Design and develop a database-driven Java application using JDBC to manage student records, enroll students in courses, perform incremental marks entry (Continuous Internal Assessment and End Semester Examination), enforce attendance eligibility rules, and generate formal academic performance reports according to Kongu Engineering College (Autonomous) - Regulation 2024.

FLOW CHART:



PROCEDURE:

- Load the MySQL Connector/J driver to enable communication between the Java application and the MySQL database.
- Establish a connection to the kec_academic database running on the local MySQL server using DriverManager.getConnection().
- Store and retrieve student details such as roll number, name, department and current semester, along with course details such as course code, title, credits and course type.
- Enroll students in semester courses, record the number of classes held and attended, and calculate the attendance percentage.
- Verify examination eligibility based on attendance. Students with 75% or above are eligible, those with 65% to 74% require medical condonation, and those below 65% are marked as Shortage of Attendance (SA).
- Record CIA-1, CIA-2, assignment, quiz and lab marks, and calculate the CIA average according to the course type.
- Record the end semester examination marks out of 100, or mark the student absent (AB), and scale the marks according to the theory, practical or embedded weightage.
- Calculate the total marks by combining the scaled CIA and ESE marks. Assign the letter grade and grade point, and mark the student as RA if the minimum pass requirements are not met.
- Generate the student mark sheet with GPA, the attendance defaulter report and the class performance summary, and display them on the command prompt console.

DATABASE SCHEMA:

1. STUDENTS TABLE:

```
mysql> DESCRIBE students;
+------------------+------------------+------+-----+---------+-------+
| Field            | Type             | Null | Key | Default | Extra |
+------------------+------------------+------+-----+---------+-------+
| roll_no          | varchar(20)      | NO   | PRI | NULL    |       |
| student_name     | varchar(100)     | NO   |     | NULL    |       |
| department       | varchar(20)      | NO   |     | NULL    |       |
| current_semester | tinyint unsigned | NO   |     | NULL    |       |
+------------------+------------------+------+-----+---------+-------+
4 rows in set (0.02 sec)
```

2. COURSES TABLE:

```
mysql> DESCRIBE courses;
+--------------+-----------------------------------------+------+-----+---------+-------+
| Field        | Type                                    | Null | Key | Default | Extra |
+--------------+-----------------------------------------+------+-----+---------+-------+
| course_code  | varchar(20)                             | NO   | PRI | NULL    |       |
| course_title | varchar(150)                            | NO   |     | NULL    |       |
| credits      | decimal(4,1)                            | NO   |     | NULL    |       |
| course_type  | enum('THEORY','PRACTICAL','EMBEDDED') | NO   |     | NULL    |       |
+--------------+-----------------------------------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

3. ENROLLMENTS TABLE:

```
mysql> DESCRIBE enrollments;
+-----------------------------+------------------+------+-----+---------+----------------+
| Field                       | Type             | Null | Key | Default | Extra          |
+-----------------------------+------------------+------+-----+---------+----------------+
| enrollment_id               | bigint           | NO   | PRI | NULL    | auto_increment |
| roll_no                     | varchar(20)      | NO   | MUL | NULL    |                |
| course_code                 | varchar(20)      | NO   | MUL | NULL    |                |
| semester                    | tinyint unsigned | NO   |     | NULL    |                |
| classes_held                | int unsigned     | NO   |     | 0       |                |
| classes_attended            | int unsigned     | NO   |     | 0       |                |
| medical_condonation_approved | tinyint(1)      | NO   |     | 0       |                |
+-----------------------------+------------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)
```

4. ASSESSMENT MARKS TABLE:

```
mysql> DESCRIBE assessment_marks;
+-----------------+--------------+------+-----+---------+----------------+
| Field           | Type         | Null | Key | Default | Extra          |
+-----------------+--------------+------+-----+---------+----------------+
| assessment_id   | bigint       | NO   | PRI | NULL    | auto_increment |
| enrollment_id   | bigint       | NO   | UNI | NULL    |                |
| cia1            | decimal(5,2) | YES  |     | NULL    |                |
| cia2            | decimal(5,2) | YES  |     | NULL    |                |
| assignment_mark | decimal(5,2) | YES  |     | NULL    |                |
| quiz_mark       | decimal(5,2) | YES  |     | NULL    |                |
| lab_mark        | decimal(5,2) | YES  |     | NULL    |                |
| ese_mark        | decimal(5,2) | YES  |     | NULL    |                |
| ese_absent      | tinyint(1)   | NO   |     | 0       |                |
+-----------------+--------------+------+-----+---------+----------------+
9 rows in set (0.00 sec)
```

SOURCE CODE:

MAINAPP.JAVA:

```
package com.kec.academic;

import java.sql.Connection;
import java.sql.SQLException;
import java.util.Scanner;

public class MainApp {

    static Scanner sc = new Scanner(System.in);

    static StudentDAO studentDAO = new StudentDAO();
    static AcademicDAO academicDAO = new AcademicDAO();

    public static void main(String[] args) {

        try (Connection con = DBConnection.getConnection()) {
            System.out.println("KEC R2024 Academic Evaluation System");
        } catch (SQLException e) {
            System.out.println("Database connection failed: " + e.getMessage());
            return;
        }

        int choice;

        do {
            System.out.println("\n========== MAIN MENU ==========");
            System.out.println("1. Register Student");
            System.out.println("2. Display Students");
            System.out.println("3. Add Course");
            System.out.println("4. Enroll Student");
            System.out.println("5. Update Attendance");
            System.out.println("6. Enter CIA Marks");
            System.out.println("7. Enter ESE Marks");
            System.out.println("8. Generate Mark Sheet");
            System.out.println("9. Attendance Defaulter Report");
            System.out.println("10. Class Performance Summary");
            System.out.println("0. Exit");

            choice = readInt("Enter your choice: ");

            try {
                switch (choice) {
                    case 1:
                        registerStudent();
                        break;
                    case 2:
                        studentDAO.displayStudents();
                        break;
                    case 3:
                        addCourse();
                        break;
                    case 4:
                        enrollStudent();
                        break;
                    case 5:
                        updateAttendance();
                        break;
                    case 6:
                        enterCiaMarks();
                        break;
                    case 7:
                        enterEseMarks();
                        break;
                    case 8:
                        generateMarkSheet();
                        break;
                    case 9:
                        academicDAO.attendanceDefaulterReport(readInt("Semester (1-8): "));
                        break;
                    case 10:
                        classSummary();
                        break;
```

```
                case 0:
                    System.out.println("Application closed.");
                    break;
                default:
                    System.out.println("Invalid choice.");
            }
        } catch (SQLException | IllegalArgumentException e) {
            System.out.println("Operation failed: " + e.getMessage());
        }

    } while (choice != 0);

    sc.close();
}

// ------------------------------------------------------ input helpers

private static String readLine(String prompt) {
    System.out.print(prompt);
    return sc.hasNextLine() ? sc.nextLine().trim() : "";
}

private static int readInt(String prompt) {
    while (true) {
        String line = readLine(prompt);
        try {
            return Integer.parseInt(line);
        } catch (NumberFormatException e) {
            System.out.println("Please enter a whole number.");
        }
    }
}

private static double readDouble(String prompt) {
    while (true) {
        String line = readLine(prompt);
        try {
            return Double.parseDouble(line);
        } catch (NumberFormatException e) {
            System.out.println("Please enter a number.");
        }
    }
}

/** Blank input means "not entered yet" and returns null. */
private static Double readOptionalDouble(String prompt) {
    while (true) {
        String line = readLine(prompt + " (blank to skip): ");
        if (line.isEmpty()) {
            return null;
        }
        try {
            return Double.parseDouble(line);
        } catch (NumberFormatException e) {
            System.out.println("Please enter a number or leave blank.");
        }
    }
}

private static boolean readYesNo(String prompt) {
    while (true) {
        String line = readLine(prompt + " (y/n): ").toLowerCase();
        if (line.equals("y") || line.equals("yes")) {
            return true;
        }
        if (line.equals("n") || line.equals("no")) {
            return false;
        }
        System.out.println("Please answer y or n.");
    }
}

// ------------------------------------------------------------ actions

private static void registerStudent() throws SQLException {
    String roll = readLine("Roll Number: ");
    String name = readLine("Student Name: ");
```

```
        String department = readLine("Department: ");
        int semester = readInt("Semester: ");
        studentDAO.addStudent(roll, name, department, semester);
    }

    private static void addCourse() throws SQLException {
        String code = readLine("Course Code: ");
        String title = readLine("Course Title: ");
        double credits = readDouble("Credits: ");
        String type = readLine("Course Type (THEORY / PRACTICAL / EMBEDDED): ");
        academicDAO.addCourse(code, title, credits, type);
    }

    private static void enrollStudent() throws SQLException {
        String roll = readLine("Roll Number: ");
        String code = readLine("Course Code: ");
        int semester = readInt("Semester: ");
        academicDAO.enrollStudent(roll, code, semester);
    }

    private static void updateAttendance() throws SQLException {
        String roll = readLine("Roll Number: ");
        String code = readLine("Course Code: ");
        int semester = readInt("Semester: ");
        int held = readInt("Classes held: ");
        int attended = readInt("Classes attended: ");
        boolean condonation = readYesNo("Medical condonation approved?");
        academicDAO.updateAttendance(roll, code, semester, held, attended, condonation);
    }

    private static void enterCiaMarks() throws SQLException {
        String roll = readLine("Roll Number: ");
        String code = readLine("Course Code: ");
        int semester = readInt("Semester: ");
        Double cia1 = readOptionalDouble("CIA 1 (0-100)");
        Double cia2 = readOptionalDouble("CIA 2 (0-100)");
        Double assignment = readOptionalDouble("Assignment (0-100)");
        Double quiz = readOptionalDouble("Quiz (0-100)");
        Double lab = readOptionalDouble("Lab (0-100)");
        academicDAO.saveCiaMarks(roll, code, semester, cia1, cia2, assignment, quiz, lab);
    }

    private static void enterEseMarks() throws SQLException {
        String roll = readLine("Roll Number: ");
        String code = readLine("Course Code: ");
        int semester = readInt("Semester: ");
        boolean absent = readYesNo("Was the student absent for ESE?");
        Double ese = absent ? null : Double.valueOf(readDouble("ESE mark (0-100): "));
        academicDAO.saveEseMarks(roll, code, semester, ese, absent);
    }

    private static void generateMarkSheet() throws SQLException {
        String roll = readLine("Roll Number: ");
        int semester = readInt("Semester: ");
        academicDAO.generateMarkSheet(roll, semester);
    }

    private static void classSummary() throws SQLException {
        String code = readLine("Course Code: ");
        int semester = readInt("Semester: ");
        academicDAO.classPerformanceSummary(code, semester);
    }
}
```

DBCONNECTION.JAVA:

```
package com.kec.academic;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL = getenvOr("DB_URL",
            "jdbc:mysql://localhost:3306/kec_academic");
```

```
    private static final String USER = getenvOr("DB_USER", "root");
    private static final String PASSWORD = getenvOr("DB_PASSWORD", "vaishu@05");

    private static String getenvOr(String key, String fallback) {
        String value = System.getenv(key);
        return (value == null || value.isEmpty()) ? fallback : value;
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    // Quick connectivity test: java -cp "out:lib/*" com.kec.academic.DBConnection
    public static void main(String[] args) {
        try (Connection con = getConnection()) {
            System.out.println("Connected to database");
        } catch (SQLException e) {
            System.out.println("Failed: " + e.getMessage());
        }
    }
}
```

GRADECALCULATOR.JAVA:

```
package com.kec.academic;
/**
 * Pure grading logic (no database access), so it can be tested on its own.
 *
 * Rules used:
 *  - CIA = average of the CIA components that have been entered (out of 100).
 *  - Theory 40 CIA + 60 ESE, Practical 60 + 40, Embedded 50 + 50.
 *  - ESE must be at least 45 (out of 100) and the overall total at least 50 to pass.
 *  - Attendance below 65%            -> SA (shortage of attendance)
 *  - Attendance 65% to below 75%     -> PENDING unless medical condonation is approved
 *  - ESE absent                      -> AB
 *  - Otherwise grade from total: O(90+) A+(80+) A(70+) B+(60+) B(50+), else RA.
 */
public class GradeCalculator {

    public static final double ESE_MIN_MARK = 45.0;
    public static final double OVERALL_MIN = 50.0;

    public static class Result {
        public final double cia;          // CIA average out of 100
        public final double total;        // weighted total out of 100
        public final String grade;
        public final int gradePoint;
        public final boolean counted;     // true when the grade is final (counts for GPA)

        Result(double cia, double total, String grade, int gradePoint, boolean counted) {
            this.cia = cia;
            this.total = total;
            this.grade = grade;
            this.gradePoint = gradePoint;
            this.counted = counted;
        }
    }

    public static double ciaAverage(Double cia1, Double cia2, Double assignment,
                                    Double quiz, Double lab) {
        double sum = 0;
        int count = 0;
        for (Double d : new Double[] {cia1, cia2, assignment, quiz, lab}) {
            if (d != null) {
                sum += d;
                count++;
            }
        }
        return count == 0 ? 0.0 : sum / count;
    }

    public static double weightedTotal(double cia, double ese, String courseType) {
        switch (courseType.toUpperCase()) {
            case "THEORY":
                return cia * 0.40 + ese * 0.60;
```

```
            case "PRACTICAL":
                return cia * 0.60 + ese * 0.40;
            case "EMBEDDED":
                return cia * 0.50 + ese * 0.50;
            default:
                throw new IllegalArgumentException("Invalid course type: " + courseType);
        }
    }

    public static Result evaluate(Double cia1, Double cia2, Double assignment,
                                  Double quiz, Double lab, Double ese,
                                  String courseType, double attendancePercentage,
                                  boolean medicalCondonationApproved, boolean eseAbsent) {

        boolean anyCia = cia1 != null || cia2 != null || assignment != null
                || quiz != null || lab != null;
        double cia = ciaAverage(cia1, cia2, assignment, quiz, lab);
        double eseMark = (ese == null) ? 0.0 : ese;
        double total = weightedTotal(cia, eseMark, courseType);

        // Attendance rules come first
        if (attendancePercentage < 65) {
            return new Result(cia, total, "SA", 0, true);
        }
        if (attendancePercentage < 75 && !medicalCondonationApproved) {
            return new Result(cia, total, "PENDING", 0, false);
        }
        if (eseAbsent) {
            return new Result(cia, total, "AB", 0, true);
        }
        if (!anyCia || ese == null) {
            return new Result(cia, total, "INCOMPLETE", 0, false);
        }

        boolean passed = eseMark >= ESE_MIN_MARK && total >= OVERALL_MIN;
        if (!passed) {
            return new Result(cia, total, "RA", 0, true);
        }

        if (total >= 90) return new Result(cia, total, "O", 10, true);
        if (total >= 80) return new Result(cia, total, "A+", 9, true);
        if (total >= 70) return new Result(cia, total, "A", 8, true);
        if (total >= 60) return new Result(cia, total, "B+", 7, true);
        return new Result(cia, total, "B", 6, true);
    }
}
```

STUDENTDAO.JAVA:

```
package com.kec.academic;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class StudentDAO {

    public void addStudent(String roll, String name, String department,
                           int semester) throws SQLException {

        if (roll == null || roll.trim().isEmpty()
                || name == null || name.trim().isEmpty()
                || department == null || department.trim().isEmpty()) {
            throw new IllegalArgumentException("Roll number, name and department are required.");
        }
        if (semester < 1 || semester > 8) {
            throw new IllegalArgumentException("Semester must be between 1 and 8.");
        }

        String sql = "INSERT INTO students "
                + "(roll_no, student_name, department, current_semester) "
                + "VALUES (?, ?, ?, ?)";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
```

```
            ps.setString(1, roll.trim().toUpperCase());
            ps.setString(2, name.trim());
            ps.setString(3, department.trim().toUpperCase());
            ps.setInt(4, semester);
            ps.executeUpdate();

            System.out.println("Student registered successfully!");
        } catch (java.sql.SQLIntegrityConstraintViolationException e) {
            throw new SQLException("Roll number " + roll.toUpperCase() + " already exists.", e);
        }
    }

    public void displayStudents() throws SQLException {

        String sql = "SELECT roll_no, student_name, department, current_semester "
                + "FROM students ORDER BY roll_no";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            System.out.println("\nSTUDENT DETAILS");
            boolean any = false;

            while (rs.next()) {
                any = true;
                System.out.println("Roll Number: " + rs.getString("roll_no"));
                System.out.println("Name: " + rs.getString("student_name"));
                System.out.println("Department: " + rs.getString("department"));
                System.out.println("Semester: " + rs.getInt("current_semester"));
                System.out.println("-------------------------");
            }

            if (!any) {
                System.out.println("No students registered yet.");
            }
        }
    }
}
```

ACADEMICDAO.JAVA:

```
package com.kec.academic;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Types;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class AcademicDAO {

    // One enrollment row joined with course, student and marks
    private static class Record {
        String roll, studentName, courseCode, courseTitle, courseType;
        double credits;
        int semester, classesHeld, classesAttended;
        boolean condonation, eseAbsent;
        Double cia1, cia2, assignment, quiz, lab, ese;

        double attendancePercentage() {
            return classesHeld == 0 ? 100.0 : (classesAttended * 100.0) / classesHeld;
        }

        GradeCalculator.Result result() {
            return GradeCalculator.evaluate(cia1, cia2, assignment, quiz, lab, ese,
                    courseType, attendancePercentage(), condonation, eseAbsent);
        }
    }

    private static final String RECORD_SELECT =
```

```
            "SELECT e.roll_no, s.student_name, e.course_code, c.course_title, "
            + "c.credits, c.course_type, e.semester, e.classes_held, "
            + "e.classes_attended, e.medical_condonation_approved, "
            + "m.cia1, m.cia2, m.assignment_mark, m.quiz_mark, m.lab_mark, "
            + "m.ese_mark, COALESCE(m.ese_absent, FALSE) AS ese_absent "
            + "FROM enrollments e "
            + "JOIN students s ON s.roll_no = e.roll_no "
            + "JOIN courses c ON c.course_code = e.course_code "
            + "LEFT JOIN assessment_marks m ON m.enrollment_id = e.enrollment_id ";

    // ------------------------------------------------------------------ helpers

    private static Double getNullableDouble(ResultSet rs, String column) throws SQLException {
        double v = rs.getDouble(column);
        return rs.wasNull() ? null : v;
    }

    private static Record readRecord(ResultSet rs) throws SQLException {
        Record r = new Record();
        r.roll = rs.getString("roll_no");
        r.studentName = rs.getString("student_name");
        r.courseCode = rs.getString("course_code");
        r.courseTitle = rs.getString("course_title");
        r.credits = rs.getDouble("credits");
        r.courseType = rs.getString("course_type");
        r.semester = rs.getInt("semester");
        r.classesHeld = rs.getInt("classes_held");
        r.classesAttended = rs.getInt("classes_attended");
        r.condonation = rs.getBoolean("medical_condonation_approved");
        r.cia1 = getNullableDouble(rs, "cia1");
        r.cia2 = getNullableDouble(rs, "cia2");
        r.assignment = getNullableDouble(rs, "assignment mark");
        r.quiz = getNullableDouble(rs, "quiz_mark");
        r.lab = getNullableDouble(rs, "lab_mark");
        r.ese = getNullableDouble(rs, "ese_mark");
        r.eseAbsent = rs.getBoolean("ese_absent");
        return r;
    }

    private static void checkMark(String label, Double mark) {
        if (mark != null && (mark < 0 || mark > 100)) {
            throw new IllegalArgumentException(label + " must be between 0 and 100.");
        }
    }

    private static void setNullableDouble(PreparedStatement ps, int index, Double v)
            throws SQLException {
        if (v == null) {
            ps.setNull(index, Types.DECIMAL);
        } else {
            ps.setDouble(index, v);
        }
    }

    /** Returns the enrollment id, or throws if the student is not enrolled. */
    private static long findEnrollmentId(Connection con, String roll, String courseCode,
                                         int semester) throws SQLException {
        String sql = "SELECT enrollment_id FROM enrollments "
                + "WHERE roll_no = ? AND course_code = ? AND semester = ?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, roll.trim().toUpperCase());
            ps.setString(2, courseCode.trim().toUpperCase());
            ps.setInt(3, semester);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return rs.getLong(1);
                }
            }
        }
        throw new IllegalArgumentException("Student " + roll.toUpperCase()
                + " is not enrolled in " + courseCode.toUpperCase()
                + " for semester " + semester + ".");
    }

    private static void checkSemester(int semester) {
        if (semester < 1 || semester > 8) {
            throw new IllegalArgumentException("Semester must be between 1 and 8.");
```

```
        }
    }

    // ------------------------------------------------------------------ courses

    public void addCourse(String code, String title, double credits, String courseType)
            throws SQLException {

        if (code == null || code.trim().isEmpty() || title == null || title.trim().isEmpty()) {
            throw new IllegalArgumentException("Course code and title are required.");
        }
        if (credits <= 0) {
            throw new IllegalArgumentException("Credits must be greater than 0.");
        }
        String type = courseType.trim().toUpperCase();
        if (!type.equals("THEORY") && !type.equals("PRACTICAL") && !type.equals("EMBEDDED")) {
            throw new IllegalArgumentException(
                    "Course type must be THEORY, PRACTICAL or EMBEDDED.");
        }

        String sql = "INSERT INTO courses (course_code, course_title, credits, course_type) "
                + "VALUES (?, ?, ?, ?)";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, code.trim().toUpperCase());
            ps.setString(2, title.trim());
            ps.setDouble(3, credits);
            ps.setString(4, type);
            ps.executeUpdate();
            System.out.println("Course added successfully!");
        } catch (java.sql.SQLIntegrityConstraintViolationException e) {
            throw new SQLException("Course " + code.toUpperCase() + " already exists.", e);
        }
    }

    // ------------------------------------------------------------------ enrollment

    public void enrollStudent(String roll, String courseCode, int semester)
            throws SQLException {

        checkSemester(semester);
        String sql = "INSERT INTO enrollments (roll_no, course_code, semester) VALUES (?, ?, ?)";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, roll.trim().toUpperCase());
            ps.setString(2, courseCode.trim().toUpperCase());
            ps.setInt(3, semester);
            ps.executeUpdate();
            System.out.println("Student enrolled successfully!");
        } catch (java.sql.SQLIntegrityConstraintViolationException e) {
            throw new SQLException("Could not enroll: student or course does not exist, "
                    + "or the student is already enrolled in this course for the semester.", e);
        }
    }

    // ------------------------------------------------------------------ attendance

    public void updateAttendance(String roll, String courseCode, int semester,
                                 int classesHeld, int classesAttended,
                                 boolean medicalCondonationApproved) throws SQLException {

        checkSemester(semester);
        if (classesHeld < 0 || classesAttended < 0) {
            throw new IllegalArgumentException("Class counts cannot be negative.");
        }
        if (classesAttended > classesHeld) {
            throw new IllegalArgumentException("Classes attended cannot exceed classes held.");
        }

        String sql = "UPDATE enrollments SET classes_held = ?, classes_attended = ?, "
                + "medical_condonation_approved = ? WHERE enrollment_id = ?";
        try (Connection con = DBConnection.getConnection()) {
            long id = findEnrollmentId(con, roll, courseCode, semester);
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, classesHeld);
                ps.setInt(2, classesAttended);
                ps.setBoolean(3, medicalCondonationApproved);
```

```
            ps.setLong(4, id);
            ps.executeUpdate();
        }
        double pct = classesHeld == 0 ? 100.0 : classesAttended * 100.0 / classesHeld;
        System.out.printf("Attendance updated: %.2f%%%n", pct);
    }
}

// ------------------------------------------------------------------ marks

public void saveCiaMarks(String roll, String courseCode, int semester,
                         Double cia1, Double cia2, Double assignment,
                         Double quiz, Double lab) throws SQLException {

    checkSemester(semester);
    checkMark("CIA 1", cia1);
    checkMark("CIA 2", cia2);
    checkMark("Assignment", assignment);
    checkMark("Quiz", quiz);
    checkMark("Lab", lab);

    String sql = "INSERT INTO assessment_marks "
            + "(enrollment_id, cia1, cia2, assignment_mark, quiz_mark, lab_mark) "
            + "VALUES (?, ?, ?, ?, ?, ?) "
            + "ON DUPLICATE KEY UPDATE cia1 = VALUES(cia1), cia2 = VALUES(cia2), "
            + "assignment mark = VALUES(assignment mark), quiz mark = VALUES(quiz mark), "
            + "lab_mark = VALUES(lab_mark)";

    try (Connection con = DBConnection.getConnection()) {
        long id = findEnrollmentId(con, roll, courseCode, semester);
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setLong(1, id);
            setNullableDouble(ps, 2, cia1);
            setNullableDouble(ps, 3, cia2);
            setNullableDouble(ps, 4, assignment);
            setNullableDouble(ps, 5, quiz);
            setNullableDouble(ps, 6, lab);
            ps.executeUpdate();
        }
        System.out.println("CIA marks saved.");
    }
}

public void saveEseMarks(String roll, String courseCode, int semester,
                         Double ese, boolean absent) throws SQLException {

    checkSemester(semester);
    if (absent) {
        ese = null;
    } else if (ese == null) {
        throw new IllegalArgumentException("ESE mark is required unless the student is
absent.");
    }
    checkMark("ESE mark", ese);

    String sql = "INSERT INTO assessment_marks (enrollment_id, ese_mark, ese_absent) "
            + "VALUES (?, ?, ?) "
            + "ON DUPLICATE KEY UPDATE ese_mark = VALUES(ese_mark), "
            + "ese_absent = VALUES(ese_absent)";

    try (Connection con = DBConnection.getConnection()) {
        long id = findEnrollmentId(con, roll, courseCode, semester);
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setLong(1, id);
            setNullableDouble(ps, 2, ese);
            ps.setBoolean(3, absent);
            ps.executeUpdate();
        }
        System.out.println("ESE marks saved.");
    }
}

// ------------------------------------------------------------------ reports

public void generateMarkSheet(String roll, int semester) throws SQLException {

    checkSemester(semester);
```

```
        String sql = RECORD_SELECT
                + "WHERE e.roll_no = ? AND e.semester = ? ORDER BY e.course_code";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, roll.trim().toUpperCase());
            ps.setInt(2, semester);

            try (ResultSet rs = ps.executeQuery()) {
                List<Record> records = new ArrayList<>();
                while (rs.next()) {
                    records.add(readRecord(rs));
                }

                if (records.isEmpty()) {
                    System.out.println("No enrollments found for " + roll.toUpperCase()
                            + " in semester " + semester + ".");
                    return;
                }

                System.out.println("\n================ MARK SHEET ================");
                System.out.println("Roll No : " + records.get(0).roll);
                System.out.println("Name    : " + records.get(0).studentName);
                System.out.println("Semester: " + semester);
                System.out.println("-------------------------------------------");
                System.out.printf("%-10s %-24s %4s %6s %6s %7s %5s %3s%n",
                        "Course", "Title", "Cr", "CIA", "Total", "Att%", "Grade", "GP");

                double creditSum = 0;
                double pointSum = 0;
                boolean pendingItems = false;

                for (Record r : records) {
                    GradeCalculator.Result res = r.result();
                    String title = r.courseTitle.length() > 24
                            ? r.courseTitle.substring(0, 24) : r.courseTitle;
                    System.out.printf("%-10s %-24s %4.1f %6.2f %6.2f %6.2f%% %5s %3d%n",
                            r.courseCode, title, r.credits, res.cia, res.total,
                            r.attendancePercentage(), res.grade, res.gradePoint);

                    if (res.counted) {
                        creditSum += r.credits;
                        pointSum += r.credits * res.gradePoint;
                    } else {
                        pendingItems = true;
                    }
                }

                System.out.println("-------------------------------------------");
                if (creditSum > 0) {
                    System.out.printf("GPA: %.2f%n", pointSum / creditSum);
                } else {
                    System.out.println("GPA: not available yet");
                }
                if (pendingItems) {
                    System.out.println("Note: PENDING / INCOMPLETE courses are not in the GPA.");
                }
                System.out.println("===========================================");
            }
        }
    }

    public void attendanceDefaulterReport(int semester) throws SQLException {

        checkSemester(semester);
        String sql = RECORD_SELECT + "WHERE e.semester = ? "
                + "AND e.classes_held > 0 "
                + "AND (e.classes_attended * 100.0 / e.classes_held) < 75 "
                + "ORDER BY e.roll_no, e.course_code";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, semester);

            try (ResultSet rs = ps.executeQuery()) {
```

```
            System.out.println("\n=== ATTENDANCE DEFAULTERS (below 75%), Semester "
                    + semester + " ===");
            System.out.printf("%-14s %-22s %-10s %7s %-12s%n",
                    "Roll No", "Name", "Course", "Att%", "Status");

            boolean any = false;
            while (rs.next()) {
                any = true;
                Record r = readRecord(rs);
                double pct = r.attendancePercentage();
                String status;
                if (pct < 65) {
                    status = "SA";
                } else if (r.condonation) {
                    status = "CONDONED";
                } else {
                    status = "PENDING";
                }
                String name = r.studentName.length() > 22
                        ? r.studentName.substring(0, 22) : r.studentName;
                System.out.printf("%-14s %-22s %-10s %6.2f%% %-12s%n",
                        r.roll, name, r.courseCode, pct, status);
            }
            if (!any) {
                System.out.println("No defaulters found.");
            }
        }
    }
}

public void classPerformanceSummary(String courseCode, int semester) throws SQLException {

    checkSemester(semester);
    String sql = RECORD_SELECT
            + "WHERE e.course_code = ? AND e.semester = ? ORDER BY e.roll_no";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, courseCode.trim().toUpperCase());
        ps.setInt(2, semester);

        try (ResultSet rs = ps.executeQuery()) {
            List<Record> records = new ArrayList<>();
            while (rs.next()) {
                records.add(readRecord(rs));
            }

            if (records.isEmpty()) {
                System.out.println("No students enrolled in " + courseCode.toUpperCase()
                        + " for semester " + semester + ".");
                return;
            }

            Map<String, Integer> distribution = new LinkedHashMap<>();
            for (String g : new String[] {"O", "A+", "A", "B+", "B", "RA", "SA", "AB",
                    "PENDING", "INCOMPLETE"}) {
                distribution.put(g, 0);
            }

            int passed = 0;
            int evaluated = 0;   // students with a final result
            double totalSum = 0;
            double highest = Double.NEGATIVE_INFINITY;
            double lowest = Double.POSITIVE_INFINITY;

            for (Record r : records) {
                GradeCalculator.Result res = r.result();
                distribution.merge(res.grade, 1, Integer::sum);

                if (res.counted) {
                    evaluated++;
                    if (res.gradePoint > 0) {
                        passed++;
                    }
                }
                boolean marksComplete = !res.grade.equals("INCOMPLETE")
```

```
                        && !res.grade.equals("AB");
                if (marksComplete && !res.grade.equals("PENDING") && !res.grade.equals("SA"))
    {
                    totalSum += res.total;
                    highest = Math.max(highest, res.total);
                    lowest = Math.min(lowest, res.total);
                }
            }

            int scored = 0;
            for (String g : new String[] {"O", "A+", "A", "B+", "B", "RA"}) {
                scored += distribution.get(g);
            }

            Record first = records.get(0);
            System.out.println("\n===== CLASS PERFORMANCE SUMMARY =====");
            System.out.println("Course  : " + first.courseCode + " - " + first.courseTitle);
            System.out.println("Semester: " + semester);
            System.out.println("Enrolled: " + records.size());
            System.out.println("Final results: " + evaluated);
            System.out.println("Passed  : " + passed);
            if (evaluated > 0) {
                System.out.printf("Pass %%  : %.2f%%%n", passed * 100.0 / evaluated);
            }
            if (scored > 0) {
                System.out.printf("Average : %.2f%n", totalSum / scored);
                System.out.printf("Highest : %.2f%n", highest);
                System.out.printf("Lowest  : %.2f%n", lowest);
            }
            System.out.println("Grade distribution:");
            for (Map.Entry<String, Integer> e : distribution.entrySet()) {
                if (e.getValue() > 0) {
                    System.out.println("  " + e.getKey() + " : " + e.getValue());
                }
            }
            System.out.println("====================================");
        }
    }
}
```

OUTPUT SCREENSHOTS:

MAIN MENU:

```
KEC R2024 Academic Evaluation System

========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
```

STUDENT & COURSE MANAGEMENT:

STUDENT REGISTRATION:

```
KEC R2024 Academic Evaluation System

========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
Enter your choice: 1
Roll Number: 25ITR166
Student Name: VAISHNAVI SM
Department: IT
Semester: 3
Student registered successfully!
```

COURSE CATALOG:

```
========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
Enter your choice: 3
Course Code: 24ITR31
Course Title: PROGRAMMING IN JAVA
Credits: 3
Course Type (THEORY / PRACTICAL / EMBEDDED): THEORY
Course added successfully!
```

COURSE ENROLLMENT:

```
========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
Enter your choice: 4
Roll Number: 25ITR166
Course Code: 24ITR31
Semester: 3
Student enrolled successfully!
```

ATTENDANCE TRACKING:

```
========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
Enter your choice: 5
Roll Number: 25ITR166
Course Code: 24ITR31
Semester: 3
Classes held: 45
Classes attended: 40
Medical condonation approved? (y/n): n
Attendance updated: 88.89%
```

MARKS ENTRY

CIA MARKS:

```
========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
Enter your choice: 6
Roll Number: 25ITR166
Course Code: 24ITR31
Semester: 3
CIA 1 (0-100) (blank to skip): 82
CIA 2 (0-100) (blank to skip): 93
Assignment (0-100) (blank to skip): 96
Quiz (0-100) (blank to skip): 88
Lab (0-100) (blank to skip): 94
CIA marks saved.
```

ESE MARKS:

```
========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
Enter your choice: 7
Roll Number: 25ITR166
Course Code: 24ITR31
Semester: 3
Was the student absent for ESE? (y/n): n
ESE mark (0-100): 89
ESE marks saved.
```

ATTENDANCE & GRADING RULES:

ATTENDANCE DEFAULTER REPORT:

```
========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
Enter your choice: 9
Semester (1-8): 3

=== ATTENDANCE DEFAULTERS (below 75%), Semester 3 ===
Roll No        Name                 Course        Att% Status
No defaulters found.
```

STUDENT SEMESTER MARK SHEET:

```
========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
Enter your choice: 8
Roll Number: 25ITR166
Semester: 3

=============== MARK SHEET ===============
Roll No : 25ITR166
Name    : VAISHNAVI SM
Semester: 3
-------------------------------------------
Course     Title                    Cr    CIA  Total    Att% Grade  GP
24ITR31    PROGRAMMING IN JAVA     3.0  90.60  89.64  88.89%    A+   9
-------------------------------------------
GPA: 9.00
===========================================
```

CLASS PERFORMANCE & RESULT SUMMARY:

```
========== MAIN MENU ==========
1. Register Student
2. Display Students
3. Add Course
4. Enroll Student
5. Update Attendance
6. Enter CIA Marks
7. Enter ESE Marks
8. Generate Mark Sheet
9. Attendance Defaulter Report
10. Class Performance Summary
0. Exit
Enter your choice: 10
Course Code: 24ITR31
Semester: 3

===== CLASS PERFORMANCE SUMMARY =====
Course  : 24ITR31 - PROGRAMMING IN JAVA
Semester: 3
Enrolled: 1
Final results: 1
Passed  : 1
Pass %  : 100.00%
Average : 89.64
Highest : 89.64
Lowest  : 89.64
Grade distribution:
  A+ : 1
=================================
```

DATABASE:

STUDENTS DATABASE:

```
mysql> USE kec_academic;
Database changed
mysql> SELECT * FROM students;
+----------+--------------+------------+------------------+
| roll_no  | student_name | department | current_semester |
+----------+--------------+------------+------------------+
| 23CS001  | Vaishnavi    | CSE        |                5 |
| 25ITR166 | VAISHNAVI SM | IT         |                3 |
+----------+--------------+------------+------------------+
2 rows in set (0.05 sec)
```

COURSES DATABASE:

```
mysql> SELECT * FROM courses;
+-------------+--------------------+---------+-------------+
| course_code | course_title       | credits | course_type |
+-------------+--------------------+---------+-------------+
| 24ITR31     | PROGRAMMING IN JAVA |     3.0 | THEORY      |
| CS101       | Data Structures    |     3.0 | THEORY      |
+-------------+--------------------+---------+-------------+
2 rows in set (0.03 sec)
```

ENROLLMENTS DATABASE:

```
mysql> SELECT * FROM enrollments;
+---------------+----------+-------------+----------+--------------+------------------+-----------------------------+
| enrollment_id | roll_no  | course_code | semester | classes_held | classes_attended | medical_condonation_approved |
+---------------+----------+-------------+----------+--------------+------------------+-----------------------------+
|             1 | 25ITR166 | 24ITR31     |        3 |           45 |               40 |                           0 |
+---------------+----------+-------------+----------+--------------+------------------+-----------------------------+
1 row in set (0.02 sec)
```

ASSESSMENT MARKS DATABASE:

```
mysql> SELECT * FROM assessment_marks;
+---------------+---------------+-------+-------+-----------------+-----------+----------+----------+------------+
| assessment_id | enrollment_id | cia1  | cia2  | assignment_mark | quiz_mark | lab_mark | ese_mark | ese_absent |
+---------------+---------------+-------+-------+-----------------+-----------+----------+----------+------------+
|             1 |             1 | 82.00 | 93.00 |           96.00 |     88.00 |    94.00 |    89.00 |          0 |
+---------------+---------------+-------+-------+-----------------+-----------+----------+----------+------------+
1 row in set (0.02 sec)
```

RESULT:

Thus, the Student Academic and Continuous Evaluation System for Kongu Engineering College (Autonomous) Regulation 2024 was successfully designed, developed and tested using Java (JDBC) and MySQL. The system automates the calculation of Continuous Internal Assessment (CIA) and End Semester Examination (ESE) marks, letter grades, attendance eligibility and the Grade Point Average (GPA), and produces the mark sheet, attendance defaulter report and class performance summary.

REFERENCES:

1. Oracle, Java SE Documentation - JDBC Overview and Architecture. https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/
2. Oracle, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
3. Oracle, MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/8.0/en/
4. GeeksforGeeks, Establishing JDBC Connection in Java. https://www.geeksforgeeks.org/establishing-jdbc-connection-in-java/
5. Tailwind CSS, Tailwind CSS Documentation. https://tailwindcss.com/docs .
6. GeeksforGeeks, Establishing JDBC Connection in Java. https://www.geeksforgeeks.org/establishing-jdbc-connection-in-java/ .
7. YouTube, Java JDBC with MySQL Tutorial Videos. https://www.youtube.com/results?search_query=Java+JDBC+MySQL+tutorial .
8. Anthropic, Claude AI Assistant (used for project ideas and guidance). https://claude.ai/ .

GITHUB:

LINK: https://github.com/vaish-0509/javaproject.git